

EzShiksha

EzShiksha is a full-stack education assistant web application designed to help students simplify learning tasks through AI-powered support. The platform combines a React frontend with an Express + MongoDB backend and Python-based processing scripts to provide features such as solving math questions from uploaded images, extracting text from images, generating notes, paraphrasing content, and checking grammar.

Overview

The project is built around the idea of making academic work faster and more accessible. Instead of manually rewriting notes or solving equations, students can upload images or text and get helpful outputs from backend processing scripts.

This repository includes:

- A React + Vite frontend in the Client folder
- An Express.js backend in the Server folder
- MongoDB integration for user authentication and profile data
- Python scripts for OCR, math solving, note generation, and grammar assistance

Features

Student-focused functionality

- User registration and login
- Cookie-based authentication with JWT
- Protected routes and user profile retrieval
- Landing page and educational marketing UI
- Math question solving from uploaded images
- Text extraction from uploaded images
- Automatic note generation from provided text
- Paraphrasing and grammar correction support
- Course, pricing, videos, and feedback sections
- Subscription/payment-related UI

Core workflows

1. Upload an image to extract text from it
2. Upload an image of a math problem to get a solution
3. Paste text to generate concise study notes
4. Paste text to paraphrase or improve grammar
5. Sign up or log in to access the app and user features

Tech Stack

Frontend

- React
- Vite
- React Router DOM
- Bootstrap / Tailwind styling support
- Axios for API calls
- React Hot Toast for notifications

Backend

- Node.js
- Express.js
- MongoDB with Mongoose
- JWT authentication
- Cookie parsing and CORS
- Multer for file uploads

AI / processing layer

- Python scripts spawned from Node
- OCR and text extraction
- Math solving logic
- Note generation and grammar enhancement logic

Project Structure

```
EzShiksha-main/
├── Client/
│   ├── public/
│   ├── src/
│   ├── package.json
│   ├── vite.config.js
│   ├── tailwind.config.js
│   └── README.md
├── Server/
│   ├── controllers/
│   ├── data/
│   ├── middlewares/
│   ├── models/
│   ├── routes/
│   ├── uploads/
│   ├── app.js
│   ├── server.js
│   ├── ForExtracting.js
│   ├── ForSolving.js
│   ├── Forgrammarly.js
│   ├── Fornotemaking.js
│   ├── Extract.py
│   ├── Grammerly.py
│   └── Notegeneration.py
```

```
├── maths.py
├── codespace.py
├── package.json
├── Testing/
│   └── notemaking.txt
├── package-lock.json
└── README.md
```

Prerequisites

Before running the project, make sure you have:

- Node.js installed
- npm installed
- MongoDB running locally or a reachable MongoDB instance
- Python installed and available in the system PATH

This project launches Python scripts from the backend using a child process, so Python must be installed for features like:

- image text extraction
- note generation
- grammar processing
- math solving

Installation

1. Clone the repository

```
git clone <repository-url>
cd EzShiksha-main
```

2. Install backend dependencies

```
cd Server
npm install
```

3. Install frontend dependencies

```
cd ../Client
npm install
```

Environment Configuration

The backend loads environment variables from `Server/.env`. Create it from the example:

```
cp Server/.env.example Server/.env
```

Set values appropriate for your environment. For example:

```
PORT=5000  
MONGO_URL=mongodb://127.0.0.1:27017  
JWT_SECRET=replace-with-a-long-random-secret  
FRONTEND_URL=http://localhost:3000  
NODE_ENV=development
```

Keep `Server/.env` private and do not commit real credentials. In particular, use a strong `JWT_SECRET` and set `MONGO_URL` to your MongoDB connection string.

Running the App

Start the backend

Open a terminal in the `Server` folder and run:

```
npm start
```

The server runs with nodemon and starts on the configured port (default: 5000).

Start the frontend

Open a second terminal in the `Client` folder and run:

```
npm run dev
```

This starts the Vite development server, usually at:

```
http://localhost:3000
```

Main Routes and Pages

Frontend pages

- `/` - Landing page
- `/login` - Login page

- `/signup` - Sign up page
- `/home` - Main dashboard/landing content after authentication
- `/divein` - Additional educational section
- `/formula` - Formula/solving section
- `/notemaking` - Note generation section
- `/paraphrase` - Paraphrasing section
- `/extract` - Image/text extraction section
- `/videos` - Video-based learning section
- `/feedback` - Feedback page
- `/subscription` - Pricing plans
- `/payment` - Payment/upgrade page

Backend API

The backend exposes these routes under `/api/v1`:

```
POST /api/v1/users/new      - Register user
POST /api/v1/users/login    - Login user
GET  /api/v1/users/me       - Get current user profile
GET  /api/v1/users/logout   - Logout current user
POST /api/v1/users/upload    - Upload image for text extraction
POST /api/v1/users/solve     - Upload image for solving math question
POST /api/v1/users/note      - Generate notes from text
POST /api/v1/users/gram      - Grammar support
```

How the Backend Works

The Express server receives requests from the frontend and then uses Python scripts through Node's child process system.

Examples:

- `/users/upload` uses `Extract.py` via `ForExtracting.js`
- `/users/solve` uses `maths.py` via `ForSolving.js`
- `/users/note` uses `Notegeneration.py` via `Fornotemaking.js`
- `/users/gram` uses `Grammerly.py` via `Forgrammerly.js`

This approach allows the project to combine JavaScript web development with Python-based processing models or scripts.

Authentication

The backend uses:

- MongoDB to store users
- bcrypt for password hashing
- JWT and cookies for session handling

- `isAuthenticated` middleware to protect routes

The Express app is configured with CORS and cookie support so the frontend can authenticate with the backend.

Notes on the Current Project State

This project appears to be a prototype or learning-focused application. Some parts are functional and others are structured as educational features with backend integrations. It is a good fit for:

- college project demos
- AI-assisted education prototypes
- learning apps focused on study support
- full-stack React + Node examples

Possible Improvements

If you want to expand the project further, good next steps include:

- adding a real OCR model or AI-powered text extraction service
- improving the math solving logic for more robust equation recognition
- integrating a real paraphrasing or grammar API
- adding dashboards, progress tracking, and lesson analytics
- creating a production-ready deployment setup
- adding tests for frontend and backend logic
- separating environment variables more cleanly for production

Troubleshooting

MongoDB connection issues

Make sure MongoDB is installed and running locally, and that your `MONGO_URL` points to the correct database.

Python execution issues

If image processing or solving routes fail, confirm that:

- Python is installed
- `python` is available in PATH on Windows
- `python3` is available on Linux/macOS

Frontend cannot reach backend

Check that:

- the backend is running on the same port configured in `FRONTEND_URL`
- CORS is configured correctly
- the frontend is pointing to the backend API URL in the app source

License

This project does not currently declare an explicit license in the repository. If you are using it for personal or educational purposes, confirm the licensing terms before commercial deployment.

Conclusion

EzShiksha is a practical full-stack application that blends frontend design, backend APIs, and Python-based educational processing tools. It is ideal for learning, demos, and further extension into a more advanced AI-powered study assistant.

If you are running this project for the first time, start with the backend and then the frontend, verify your MongoDB connection, and ensure Python is available on your system.